Reviewer Pack — Minimal Order of Formal Groups and Communication Complexity …

Entry ddfbed2a2c60 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 05:38:44 UTC

Minimal Order of Formal Groups and Communication Complexity Rank Correlation

Entry ID: `ddfbed2a2c60` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 05:38:44 UTC

1. Conjecture

Field A (mathematical branch): Formal Group Theory **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For all Boolean functions f, the minimal order of an associated formal group G_f is linearly correlated with the rank variance of f's communication matrix, such that |order(G_f)| = Θ(rank_variance(f)).

Rationale (proposer's reasoning):

> Formal groups provide a categorical framework for studying algebraic structures. If the conjecture holds, it suggests a deep connection between algebraic structure and communication complexity, potentially revealing new insights into lower bounds.

Taxonomy category: `FormalGroupCommunicationComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a052a258a8f2838b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the Boolean functions f, for n ≤ 40, exhibit a correlation coefficient between the order of G_f and its rank variance greater than or equal to 0.8, with no seed producing a correlation coefficient less than -0.8.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `formal group theory AND communication complexity rank` - `matrix rank in communication complexity AND formal groups` - `correlation between order of formal groups and communication matrix rank variance`

Top relevant hits considered: - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2 - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/2403.16671v5] Twisted conjugacy in dihedral Artin groups I: Torus Knot groups - [http://arxiv.org/abs/2506.23031v2] Andrews-Curtis groups - [http://arxiv.org/abs/2305.09818v2] Groups of F-Type - [http://arxiv.org/abs/1906.03766v3] Variance Reduction in Gradient Exploration for Online Learning to Rank - [http://arxiv.org/abs/1111.5447v1] Special rank one groups are perfect

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=25.9s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_matrix(f):
        n = len(f)
        matrix = [[0] * (2**(n-1)) for _ in range(2**(n-1))]
        for i in range(2**(n-1)):
            for j in range(2**(n-1)):
                input1 = [i >> k & 1 for k in range(n)]
                input2 = [j >> k & 1 for k in range(n)]
                output1 = f[input1.index(0) * 2 + input1.index(1)]
                output2 = f[input2.index(0) * 2 + input2.index(1)]
                matrix[i][j] = (output1, output2)
        return matrix

    def rank_variance(matrix):
        n = len(matrix)
        total = sum(sum(row) for row in matrix)
        mean = Fraction(total, n**2)
        variance = sum((sum(row) - mean)**2 for row in matrix) / n
```

```
        return variance

    def formal_group_order(f):
        # Placeholder for actual implementation of formal group order calculation
        return len(f)

    instances_tested = 0
    total_correlation = 0.0
    max_n = 1

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            f = generate_boolean_function(n)
            matrix = communication_matrix(f)
            variance = rank_variance(matrix)
            order = formal_group_order(f)
            if order == 0:
                continue
            correlation = Fraction(variance * order).limit_denominator()
            total_correlation += correlation
            instances_tested += 1
            max_n = max(max_n, n)

    mean_correlation = total_correlation / instances_tested if instances_tested > 0 else 0
    conjecture_holds = -0.8 <= mean_correlation <= 0.8

    return {
        "metric_name": "correlation",
        "metric_value": float(mean_correlation),
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)).sqrt()
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(r["metric_value"] < -0.8 or r["metric_value"] > 0.8 for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not (-0.8 <= r["metric_valu
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Re-run the test with increased robustness and error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13360
2	preregistration	ollama_remote	glm4:latest	0	0	14205
3	novelty	ollama_remote	glm4:latest	0	0	8530
4	novelty	ollama_remote	glm4:latest	0	0	11932
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23292
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31930
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20430
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13607
9	judge	ollama_remote	glm4:latest	0	0	71601

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 208887 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ddfbed2a2c60.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ddfbed2a2c60.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ddfbed2a2c60.tar.gz` (if generated)